

SUBMITTED BY:

NAME	CH.V.S.ROHITH
ROLL NO	CSE25024
YEAR / SEM / SECTION	2 / 3 / CSE-A
CREDITS	5

SUBMITTED TO:

NAME	Dr.B.RAJ KUMAR
DEPARTMENT	SCHOOL OF COMPUTING
DESIGNATION	ASST. PROFFESOR (SI grade)

S NO:	TITLE	DATE	REMARKS
1	WEEK 1: 1a: Recursive countdown from n 1b: Recursive calculation of compound growth factor 1c: Recursive search for emo id in list 1d: Recursive factorial calculation 1e: Recursive fibonacci series	23-07-26	
2	WEEK 2: 2a: Linear Search Program 2b: Binary Search Program	30-07-27	
3	WEEK 3: 3a: Bubble Sort 3b: Insertion Sort 3c: Selection Sort 3d: Merge Sort 3e: Quick Sort	06-08-26	
4	WEEK 4: 4a: Singly Linked List 4b: Doubly Linked List 4c: Circular Linked List	13-08-26	
5	WEEK 5: 5a: Stack Using Array 5b: Stack Using Linked List	27-08-26	
6	WEEK 6: 6a: Queue Using Array 6b: Queue Using Linked List 6c: Circular Queue	03-09-26	
7	WEEK 7: 7a: Binary tree using arrays 7b: Binary tree using linked list	24-09-26	
8	WEEK 8: 8a: Binary search tree using arrays 8b: Binary search tree using linked list	08-10-26	
9	WEEK 9: 9a: AVL tree using arrays 9b: AVL tree using linked list	22-10-26	
10	WEEK 10: 10a: Hashing using separate chaining 10b: Hashing using linear probing 10c: Hashing using quadratic probing	29-10-26	

WEEK-1

1A)

AIM:

A rocket launch system displays a countdown. Use no loops. Write a Python program to display the countdown from n to 1, followed by “Launch”.

PROGRAM:

```
def countdown(n):
    while n>0:
        print(n)
        n-=1
    print(n)
    print("Launch")


n=int(input("enter number:"))
countdown(n)
```

OUTPUT:

```
========== RESTART: D:/CH ROHITH/DSA(CSE25024)/1A.py ==========
enter number:10
10
9
8
7
6
5
4
3
2
1
0
Launch
```

EXPLANATION:

- Defines a recursive countdown() function that prints numbers in decreasing order.
- Prints the current value of n before making the recursive call.
- When n becomes 0, the recursion stops.
- Otherwise, the function calls itself with n-1.
- Takes the starting number as input and displays the countdown.

ERROR TABLE:

S.No	Error Obtained	Rectification
1	Countdown does not stop	Check the condition when n reaches 0
2	Program stops for a very large number	Use a loop instead of recursion
3	Negative numbers are printed unexpectedly	Allow only non negative starting values

1B)

AIM:

Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate p^n (power), where p = principal growth factor and n = number of years.

PROGRAM:

```
def power(p, n):
    if n == 0:
        return 1
    return p * power(p, n - 1)

p = float(input("Enter prinicipal amount: "))
n = int(input("Enter number of years: "))

print("p^n =", power(p, n))
```

OUTPUT:

```
========== RESTART: D:/CH ROHITH/DSA(CSE25024)/WEEK 1/1B.py ==========
Enter prinicipal amount: 100
Enter number of years: 2
p^n = 10000.0
```

EXPLANATION:

- Defines a recursive power() function to calculate \(p^n\).
- Returns 1 when n <= 0, which is the base case.
- Otherwise, multiplies p by power(p, n-1) repeatedly.
- Takes the base (p) and exponent (n) as input.
- Calls the function and prints the calculated power.

ERROR TABLE:

S.No	Error Obtained	Rectification
1	Incorrect result for negative exponent	Add separate handling for negative exponents
2	Program becomes slow for large exponent	Use an efficient power calculation method
3	Wrong result for exponent zero	Make sure the base case returns 1

1C)

AIM:

An HR department stores employee IDs in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

PROGRAM:

```
def search(A,k,n=0):
    if n>=len(A):
        return -1
    if A[n]==k:
        return n
    return search(A,k,n+1)

A=[1,2,3,4,5,6,7,8]
print(A)
k=int(input("enter num to search:"))

print(search(A,k))
```

OUTPUT:

```
=============== RESTART: D:/CH ROHITH/DSA(CSE25024)/WEEK 1/1C.py ===============
[1, 2, 3, 4, 5, 6, 7, 8]
enter num to search:3
2
```

EXPLANATION:

- Defines a recursive search() function to find the target element in the list.
- Checks each element starting from index 0 using recursion.
- If the target is found, its index is returned.
- If the end of the list is reached, -1 is returned.
- Takes the target value as input and displays the search result.

ERROR TABLE:

S.No	Error Obtained	Rectification
1	Target element is not found	Check whether the target exists in the list
2	Recursion stops before checking all elements	Verify the index increment in the recursive call
3	Program stops for a large list	Use an iterative search method

1D)

AIM:

In a logistics company, the number of possible ways to arrange n different parcels is calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number.

PROGRAM:

```
def factorial(n):
    if n<=0:
        return 1
    else:
        return n*factorial(n-1)

n=int(input("Enter number:"))
print(factorial(n))
```

OUTPUT:

```
=============== RESTART: D:/CH ROHITH/DSA(CSE25024)/WEEK 1/1D.py ===============
Enter number:24
620448401733239439360000
```

EXPLANATION:

- Defines a recursive factorial() function to calculate the factorial of a number.
- Returns 1 when n <= 0, which also handles 0!.
- Otherwise, multiplies n by factorial(n-1) until reaching the base case.
- Takes a number as input from the user.
- Calls the function and prints the factorial result.

ERROR TABLE:

S.No	Error Obtained	Rectification
1	Incorrect factorial for negative input	Restrict the input to non negative numbers
2	Program stops for a large number	Use an iterative approach
3	Wrong factorial result	Check the recursive multiplication and base condition

1E)

AIM:

Write a recursive Python program to print the first n terms of the Fibonacci series.

PROGRAM:

```
def fibonacci(n):
    if n<=0:
        return 0
    elif n==1:
        return 1
    else:
        return fibonacci(n-1)+fibonacci(n-2)


n= int(input("Enter number of terms:"))
for i in range(n):
    print(fibonacci(i))
```

OUTPUT:

```
=============== RESTART: D:/CH ROHITH/DSA(CSE25024)/WEEK 1/1E.py ===============
Enter number of terms:6
0
1
1
2
3
5
```

EXPLANATION:

- Defines a recursive fibonacci() function to calculate Fibonacci numbers.
- Returns 0 for n <= 0 and 1 for n == 1.
- For other values, it recursively adds the previous two Fibonacci numbers.
- Takes the number of terms from the user.
- Uses a loop to generate and print each Fibonacci term.
- The sequence starts as 0, 1, 1, 2, 3, 5, ...

ERROR TABLE:

S.No	Error Obtained	Rectification
1	Program runs very slowly	Use an iterative or optimized approach
2	Incorrect sequence is displayed	Check the base conditions for 0 and 1
3	Program stops for large input	Avoid excessive recursive calls

WEEK-2

2A)

AIM:

Write a program to perform linear search on a given input.

PROGRAM:

```
def linearSearch(a,key):
    for i in range(n):
        if a[i]==key:
            return i
    return -1

n=int(input("Enter number of elements:"))
a=[]
for i in range(n):
    x=int(input("Enter elements:"))
    a.append(x)
key=int(input("Enter element to Find:"))
print(linearSearch(a,key))
```

OUTPUT:

```
=============== RESTART: D:/CH ROHITH/DSA(CSE25024)/WEEk 2/2A.py ===============
Enter number of elements:5
Enter elements:1
Enter elements:7
Enter elements:18
Enter elements:45
Enter elements:93
Enter element to Find:45
3
```

EXPLANATION:

- Takes the number of elements and stores them in a list.
- Takes the target element to be searched.
- linearSearch() checks each element one by one.
- If the target is found, its index is returned.
- If not found, -1 is returned.
- The result is displayed using print().

ERROR TABLE:

S.No	Error Obtained	Rectification
1	Element is not found	Check whether the required element is present in the list
2	Wrong index is displayed	Check the list indexing starting from 0
3	Search stops unexpectedly	Verify the range used in the for loop

2B)

AIM:

Write a Python program to perform binary search on the given input. Verify by taking sorted array & unsorted array as input.

PROGRAM:

```
def binary_search(arr, key):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1

    return -1

n = int(input("Enter number of elements: "))
arr = []

for i in range(n):
    arr.append(int(input("Enter element: ")))

key = int(input("Enter element to search: "))

if arr == sorted(arr):
    result = binary_search(arr, key)

    if result != -1:
        print("Element found at index", result)
```

```
    else:
        print("Element not found")
else:
    print("Array is unsorted. Binary search cannot be performed.")
```

OUTPUT:

```
=============== RESTART: D:/CH ROHITH/DSA(CSE25024)/WEEk 2/2B.py ===============
Enter number of elements: 5
Enter element: 1
Enter element: 7
Enter element: 18
Enter element: 45
Enter element: 93
Enter element to search: 18
Element found at index 2
```

EXPLANATION:

- Takes the number of elements and stores them in a list.
- Sorts the list using a.sort() because binary search requires sorted data.
- Calculates the middle element and compares it with the target.
- If the target is smaller or larger, searches only the appropriate half.
- Returns the index if found; otherwise returns -1.
- Displays whether the element is found or not.

ERROR TABLE:

S.No	Error Obtained	Rectification
1	Wrong element is returned	Make sure the list is sorted before searching
2	Search gives incorrect index	Check the calculation of the middle index
3	Element is not found	Verify the low and high boundaries

WEEK-3

3A)

Aim:

Write a python program to sort an array using selection sort and take the elements as input from user

PROGRAM:

```
n = int(input("Enter number of elements: "))
arr = []

for i in range(n):
    element = int(input("Enter element: "))
    arr.append(element)

for i in range(n - 1):
    min_index = i
    for j in range(i + 1, n):
        if arr[j] < arr[min_index]:
            min_index = j

    arr[i], arr[min_index] = arr[min_index], arr[i]

print("Sorted array:", arr)
```

OUTPUT:

```
========= RESTART: D:/CH ROHITH/DSA/WEEK 3 (Sorting)/selection sort.py ====
Enter number of elements: 5
Enter element: 3
Enter element: 5
Enter element: 2
Enter element: 3
Enter element: 5
Sorted array: [2, 3, 3, 5, 5]
```

Explanation:

- Take the number of elements from the user.
- Created an empty list arr.

- Used a for loop to take each element individually.
- min_index stores the position of the smallest element.
- The inner loop searches for the smallest element in the unsorted portion.
- Swapped the smallest element with the first unsorted element.
- Repeated this until the array was sorted

Error Table:

S.No.	Errors Observed	Rectification
1)	Incorrect initialization of min_index, causing the minimum element to be identified incorrectly.	Initialize min_index = i before searching for the minimum element.
2)	Starting the inner loop from 0, causing already sorted elements to be checked again.	Start the inner loop from i + 1.
3)	Forgetting to update min_index when a smaller element is found.	Use min_index = j inside the if condition.
4)	Incorrect swapping of elements, resulting in an unsorted array.	Use arr[i], arr[min_index] = arr[min_index], arr[i].
5)	Invalid (non-numeric) input entered by the user.	Enter a valid integer for each array element.
6)	Incorrect indentation causing an IndentationError.	Maintain proper indentation inside for loops and if statements.

3B)

Aim:

Write a python program to sort an array using Insertion sort and take the elements as input from user

PROGRAM:

```
n = int(input("Enter number of elements: "))
arr = []

for i in range(n):
    element = int(input("Enter element: "))
    arr.append(element)

for i in range(1, n):
    key = arr[i]
    j = i - 1

    while j >= 0 and arr[j] > key:
        arr[j + 1] = arr[j]
        j = j - 1
```

```
    arr[j + 1] = key

print("Sorted array:", arr)
```

OUTPUT:

```
========= RESTART: D:/CH ROHITH/DSA/WEEK 3 (Sorting)/Insertion Sort.py =======
Enter number of elements: 4
Enter element: 232
Enter element: 32
Enter element: 41
Enter element: 412
Sorted array: [32, 41, 232, 412]
```

Explanation:

- Took the number of elements from the user.
- Used a loop to input every element.
- Started sorting from index 1.
- Stored the current element in key.
- Compared key with the elements before it.
- Shifted larger elements one position to the right.
- Inserted key into its correct position.
- Repeated until the array was sorted.

Error Table:

S.No.	Errors Observed	Rectification
1)	Starting the outer loop from 0, unnecessarily processing the first element.	Start the loop from 1 because the first element is already considered sorted.
2)	Forgetting to store the current element in key.	Use key = arr[i] before shifting elements.
3)	Forgetting j -= 1, causing the while loop to run indefinitely.	Decrease j by using j -= 1 after shifting an element.
4)	Placing the key at arr[j] instead of arr[j + 1].	Insert the key using arr[j + 1] = key.
5)	Incorrect condition in the while loop.	Use j >= 0 and arr[j] > key to shift larger elements.

3C)

Aim:

Write a python program to sort an array using Bubble sort and take the elements as input from user

PROGRAM:

```
n = int(input("Enter number of elements: "))

arr = []

for i in range(n):
    element = int(input("Enter element: "))
    arr.append(element)

for i in range(n - 1):
    for j in range(n - 1 - i):
        if arr[j] > arr[j + 1]:
            # Swap
            arr[j], arr[j + 1] = arr[j + 1], arr[j]

print("Sorted array:", arr)
```

OUTPUT:

```
========== RESTART: D:/CH ROHITH/DSA/WEEK 3 (Sorting)/Bubble Sort.py ==========
Enter number of elements: 4
Enter element: 65
Enter element: 62
Enter element: 24
Enter element: 54
Sorted array: [24, 54, 62, 65]
```

Explanation:

- Took the number of elements.
- Input each element using a loop.
- Compared two adjacent elements.
- If the left element was greater, we swapped them.
- After every pass, the largest remaining element moved to the end.
- Reduced the comparison range after every pass.
- Repeated until the array became sorted.

Error Table:

S.No.	Errors Observed	Rectification
1)	Comparing non-adjacent elements instead of neighboring elements.	Compare arr[j] with arr[j + 1].
2)	Using an incorrect inner loop range.	Use range(n - 1 - i) so that already sorted elements are not checked
3)	Forgetting to swap elements when they are in the wrong order.	Use arr[j], arr[j + 1] = arr[j + 1], arr[j].
4)	Using < instead of > for ascending order.	Use if arr[j] > arr[j + 1] for ascending sorting.
5)	Using an incorrect outer loop range.	Use range(n - 1) to perform the required passes.
6)	Entering non-numeric values as array elements.	Enter valid integer values using int(input()).

WEEK-4

4A)

Aim:

Write a python programme to implement a single linked list which could perform the below operations ,by taking an input from the user.
1)create a linked list
2)Insert at beginning
3)Insert at end
4)insert at index
5)Delete by value
6)Delete First Node
7)Delete Last Node
8)Count number of nodes
9)Display/Traverse
10)Exit

PROGRAM:

```
# Singly Linked List Implementation

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class SinglyLinkedList:
    def __init__(self):
        self.head = None

    # 1. Create Linked List
    def create(self):
        n = int(input("Enter number of nodes: "))

        self.head = None
        temp = None

        for i in range(n):
            data = int(input(f"Enter data for node {i + 1}: "))
            new_node = Node(data)

            if self.head is None:
                self.head = new_node
                temp = new_node
            else:
```

```
            temp.next = new_node
            temp = new_node

        print("Linked list created successfully.")

    # 2. Insert at Beginning
    def insert_beginning(self):
        data = int(input("Enter value: "))
        new_node = Node(data)

        new_node.next = self.head
        self.head = new_node

        print("Node inserted at beginning.")

    # 3. Insert at End
    def insert_end(self):
        data = int(input("Enter value: "))
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            print("Node inserted at end.")
            return

        temp = self.head

        while temp.next is not None:
            temp = temp.next

        temp.next = new_node

        print("Node inserted at end.")

    # 4. Insert at Index
    def insert_at_index(self):
        index = int(input("Enter index: "))
        data = int(input("Enter value: "))

        if index < 0:
            print("Invalid index.")
            return

        # Insert at beginning
        if index == 0:
            new_node = Node(data)
            new_node.next = self.head
            self.head = new_node
            print("Node inserted.")
            return
```

```
        temp = self.head

        for i in range(index - 1):
            if temp is None:
                print("Invalid index.")
                return
            temp = temp.next

        if temp is None:
            print("Invalid index.")
            return

        new_node = Node(data)
        new_node.next = temp.next
        temp.next = new_node

        print("Node inserted at index", index)

    # 5. Delete by Value
    def delete_by_value(self):
        value = int(input("Enter value to delete: "))

        if self.head is None:
            print("Linked list is empty.")
            return

        # If first node contains the value
        if self.head.data == value:
            self.head = self.head.next
            print("Node deleted.")
            return

        temp = self.head

        while temp.next is not None:
            if temp.next.data == value:
                temp.next = temp.next.next
                print("Node deleted.")
                return

            temp = temp.next

        print("Value not found.")

    # 6. Delete First Node
    def delete_first(self):
        if self.head is None:
            print("Linked list is empty.")
            return
```

```
        self.head = self.head.next
        print("First node deleted.")

    # 7. Delete Last Node
    def delete_last(self):
        if self.head is None:
            print("Linked list is empty.")
            return

        # Only one node
        if self.head.next is None:
            self.head = None
            print("Last node deleted.")
            return

        temp = self.head

        while temp.next.next is not None:
            temp = temp.next

        temp.next = None

        print("Last node deleted.")

    # 8. Count Number of Nodes
    def count_nodes(self):
        count = 0
        temp = self.head

        while temp is not None:
            count += 1
            temp = temp.next

        print("Number of nodes:", count)

    # 9. Display / Traverse
    def display(self):
        if self.head is None:
            print("Linked list is empty.")
            return

        temp = self.head

        print("Linked List:", end=" ")

        while temp is not None:
            print(temp.data, end=" -> ")
            temp = temp.next
```

```
        print("None")


# Main Program

linked_list = SinglyLinkedList()

while True:
    print("\n========== SINGLY LINKED LIST ==========")
    print("1. Create Linked List")
    print("2. Insert at Beginning")
    print("3. Insert at End")
    print("4. Insert at Index")
    print("5. Delete by Value")
    print("6. Delete First Node")
    print("7. Delete Last Node")
    print("8. Count Number of Nodes")
    print("9. Display / Traverse")
    print("10. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        linked_list.create()

    elif choice == 2:
        linked_list.insert_beginning()

    elif choice == 3:
        linked_list.insert_end()

    elif choice == 4:
        linked_list.insert_at_index()

    elif choice == 5:
        linked_list.delete_by_value()

    elif choice == 6:
        linked_list.delete_first()

    elif choice == 7:
        linked_list.delete_last()

    elif choice == 8:
        linked_list.count_nodes()

    elif choice == 9:
        linked_list.display()

    elif choice == 10:
```

```
        print("Program exited.")
        break

    else:
        print("Invalid choice. Please try again.")
```

OUTPUT:

```
===== RESTART: D:/CH ROHITH/DSA/WEEK 4 (Linked Lists)/singly linked list.py ====

========== SINGLY LINKED LIST ==========
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 1
Enter number of nodes: 2
Enter data for node 1: 2
Enter data for node 2: 4
Linked list created successfully.

========== SINGLY LINKED LIST ==========
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 9
Linked List: 2 -> 4 -> None
```

Explanation:

- Defines Node and SLL classes to create and manage a singly linked list.
- The create function creates nodes and connects them using the next pointer.
- Provides operations to insert nodes at the beginning, end, and a specific index.
- Provides operations to delete a node by value, from the beginning, and from the end.
- The count and display functions show the number of nodes and the contents of the list.
- A menu driven loop allows the user to select and perform different linked list operations.

Error Table:

S.No.	Errors Observed	Rectification
1)	IndentationError in class or function definitions	Maintain proper indentation inside classes and functions
2)	AttributeError while accessing next	Check that the node is created correctly before accessing next
3)	Index out of range during insertion	Enter an index within the valid linked list range

4B)

Aim:

Write a python program to implement a Double linked list which could perform the below operations ,by taking an input from the user.
1)Insert at beginning
2)Insert at end
3)insert at index
4)Delete by value
5)Delete First Node
6)Delete Last Node
7)Display the Forward Nodes
8)Display the Backward Nodes
9)Exit

PROGRAM:

```
# Doubly Linked List Implementation

class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None

class DoublyLinkedList:

    def __init__(self):
        self.head = None

    # 1. Insert at Beginning
    def insert_beginning(self):
        data = int(input("Enter value: "))
```

```
    new_node = Node(data)

    if self.head is None:
       self.head = new_node
    else:
        new_node.next = self.head
        self.head.prev = new_node
        self.head = new_node

    print("Node inserted at beginning.")

# 2. Insert at End
def insert_end(self):
    data = int(input("Enter value: "))
    new_node = Node(data)

    if self.head is None:
        self.head = new_node
        print("Node inserted at end.")
        return
    temp = self.head

    while temp.next is not None:
        temp = temp.next

    temp.next = new_node
    new_node.prev = temp

    print("Node inserted at end.")

# 3. Insert at Index
def insert_at_index(self):
    index = int(input("Enter index: "))
    data = int(input("Enter value: "))

    if index < 0:
        print("Invalid index.")
        return

    # Insert at beginning
    if index == 0:
        new_node = Node(data)

        new_node.next = self.head

        if self.head is not None:
            self.head.prev = new_node

        self.head = new_node
```

```
            print("Node inserted.")
            return

        temp = self.head

        # Move to node before the required index
        for i in range(index - 1):
            if temp is None:
                print("Invalid index.")
                return
            temp = temp.next

        if temp is None:
            print("Invalid index.")
            return

        new_node = Node(data)

        new_node.next = temp.next
        new_node.prev = temp

        if temp.next is not None:
            temp.next.prev = new_node

        temp.next = new_node

        print("Node inserted at index", index)

    # 4. Delete by Value
    def delete_by_value(self):
        value = int(input("Enter value to delete: "))

        if self.head is None:
            print("Linked list is empty.")
            return

        temp = self.head

        while temp is not None:

            if temp.data == value:

                # If deleting first node
                if temp.prev is None:
                    self.head = temp.next

                    if self.head is not None:
                        self.head.prev = None

                # If deleting any other node
```

```
            else:
                temp.prev.next = temp.next

                if temp.next is not None:
                    temp.next.prev = temp.prev

                print("Node deleted.")
                return

            temp = temp.next

        print("Value not found.")

    # 5. Delete First Node
    def delete_first(self):
        if self.head is None:
            print("Linked list is empty.")
            return

        self.head = self.head.next

        if self.head is not None:
            self.head.prev = None

        print("First node deleted.")

    # 6. Delete Last Node
    def delete_last(self):
        if self.head is None:
            print("Linked list is empty.")
            return

        temp = self.head

        # Only one node
        if temp.next is None:
            self.head = None
            print("Last node deleted.")
            return

        while temp.next is not None:
            temp = temp.next

        temp.prev.next = None

        print("Last node deleted.")

    # 7. Display Forward Nodes
    def display_forward(self):
        if self.head is None:
```

```
            print("Linked list is empty.")
            return

        temp = self.head

        print("Forward: ", end="")

        while temp is not None:
            print(temp.data, end=" <-> ")
            temp = temp.next

        print("None")

    # 8. Display Backward Nodes
    def display_backward(self):
        if self.head is None:
            print("Linked list is empty.")
            return

        temp = self.head

        # Move to last node
        while temp.next is not None:
            temp = temp.next

        print("Backward: ", end="")

        while temp is not None:
            print(temp.data, end=" <-> ")
            temp = temp.prev

        print("None")

# Main Program

dll = DoublyLinkedList()

while True:
    print("\n========== DOUBLY LINKED LIST ==========")
    print("1. Insert at Beginning")
    print("2. Insert at End")
    print("3. Insert at Index")
    print("4. Delete by Value")
    print("5. Delete First Node")
    print("6. Delete Last Node")
    print("7. Display Forward Nodes")
    print("8. Display Backward Nodes")
    print("9. Exit")

    choice = int(input("Enter your choice: "))
```

```
    if choice == 1:
        dll.insert_beginning()

    elif choice == 2:
        dll.insert_end()

    elif choice == 3:
        dll.insert_at_index()

    elif choice == 4:
        dll.delete_by_value()

    elif choice == 5:
        dll.delete_first()

    elif choice == 6:
        dll.delete_last()

    elif choice == 7:
        dll.display_forward()

    elif choice == 8:
        dll.display_backward()

    elif choice == 9:
        print("Program exited.")
        break

    else:
        print("Invalid choice. Please try again.")
```

OUTPUT:

```
===== RESTART: D:/CH ROHITH/DSA/WEEK 4 (Linked Lists)/Doubly Linked List.py ====

========== DOUBLY LINKED LIST ==========
1. Insert at Beginning
2. Insert at End
3. Insert at Index
4. Delete by Value
5. Delete First Node
6. Delete Last Node
7. Display Forward Nodes
8. Display Backward Nodes
9. Exit
Enter your choice: 1
Enter value: 24
Node inserted at beginning.

========== DOUBLY LINKED LIST ==========
1. Insert at Beginning
2. Insert at End
3. Insert at Index
4. Delete by Value
5. Delete First Node
6. Delete Last Node
7. Display Forward Nodes
8. Display Backward Nodes
9. Exit
Enter your choice: 8
```

Explanation:

- Defines a Node class with data, previous link and next link for a doubly linked list.
- Provides functions to insert nodes at the beginning, end and a specific index.
- Provides functions to delete nodes from the beginning, end and a specific index.
- Maintains both prev and next links while inserting and deleting nodes.
- The display functions show the list in forward and backward directions.
- A menu allows the user to perform different doubly linked list operations.

Error Table:

S.No.	Errors Observed	Rectification
1)	Invalid Index during insertion	Enter an index within the valid list range
2)	Links are not updated correctly after deletion	Update both prev and next links properly
3)	Backward display gives incorrect order	Traverse to the last node before moving backward

4C)

Aim:

Write a python program to implement a Circular linked list which could perform the below operations ,by taking an input from the user.
1)create a linked list
2)Insert at beginning
3)Insert at end
4)insert at index
5)Delete by value
6)Delete First Node
7)Delete Last Node
8)Count number of nodes
9)Display
  a)Head to Tail
  b)Tail to Head
10)Exit

PROGRAM:

```
# Circular Linked List Implementation

class Node:
   def __init__(self, data):
      self.data = data
      self.next = None
```

```
class CircularLinkedList:

    def __init__(self):
        self.head = None

    # 1. Create Linked List
    def create(self):
        n = int(input("Enter number of nodes: "))

        self.head = None
        tail = None

        for i in range(n):
            data = int(input(f"Enter data for node {i + 1}: "))
            new_node = Node(data)

            if self.head is None:
                self.head = new_node
                tail = new_node
                new_node.next = self.head
            else:
                tail.next = new_node
                tail = new_node
                tail.next = self.head

        print("Circular linked list created successfully.")

    # 2. Insert at Beginning
    def insert_beginning(self):
        data = int(input("Enter value: "))
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            new_node.next = self.head
            print("Node inserted at beginning.")
            return

        tail = self.head

        while tail.next != self.head:
            tail = tail.next

        new_node.next = self.head
        self.head = new_node
        tail.next = self.head

        print("Node inserted at beginning.")
```

```
# 3. Insert at End
def insert_end(self):
    data = int(input("Enter value: "))
    new_node = Node(data)

    if self.head is None:
        self.head = new_node
        new_node.next = self.head
        print("Node inserted at end.")
        return

    tail = self.head

    while tail.next != self.head:
        tail = tail.next

    tail.next = new_node
    new_node.next = self.head

    print("Node inserted at end.")

# 4. Insert at Index
def insert_at_index(self):
    index = int(input("Enter index: "))
    data = int(input("Enter value: "))

    if index < 0:
        print("Invalid index.")
        return

    # Insert at beginning
    if index == 0:
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            new_node.next = self.head
        else:
            tail = self.head

            while tail.next != self.head:
                tail = tail.next

            new_node.next = self.head
            self.head = new_node
            tail.next = self.head

        print("Node inserted.")
        return
```

```
    if self.head is None:
        print("Invalid index.")
        return

    temp = self.head
    current_index = 0

    # Find node before the required index
    while current_index < index - 1:
        temp = temp.next
        current_index += 1

        if temp == self.head:
            print("Invalid index.")
            return

    new_node = Node(data)

    new_node.next = temp.next
    temp.next = new_node

    print("Node inserted at index", index)

# 5. Delete by Value
def delete_by_value(self):
    value = int(input("Enter value to delete: "))

    if self.head is None:
        print("Linked list is empty.")
        return

    # If only one node
    if self.head.next == self.head:
        if self.head.data == value:
            self.head = None
            print("Node deleted.")
        else:
            print("Value not found.")
        return

    # If head contains the value
    if self.head.data == value:
        tail = self.head

        while tail.next != self.head:
            tail = tail.next

        self.head = self.head.next
        tail.next = self.head
```

```
            print("Node deleted.")
            return

        # Search for the value
        temp = self.head

        while temp.next != self.head:
            if temp.next.data == value:
                temp.next = temp.next.next
                print("Node deleted.")
                return

            temp = temp.next

        print("Value not found.")

    # 6. Delete First Node
    def delete_first(self):
        if self.head is None:
            print("Linked list is empty.")
            return

        # Only one node
        if self.head.next == self.head:
            self.head = None
            print("First node deleted.")
            return

        tail = self.head

        while tail.next != self.head:
            tail = tail.next

        self.head = self.head.next
        tail.next = self.head

        print("First node deleted.")

    # 7. Delete Last Node
    def delete_last(self):
        if self.head is None:
            print("Linked list is empty.")
            return

        # Only one node
        if self.head.next == self.head:
            self.head = None
            print("Last node deleted.")
            return
```

```
        temp = self.head

        # Find second-last node
        while temp.next.next != self.head:
            temp = temp.next

        temp.next = self.head

        print("Last node deleted.")

    # 8. Count Number of Nodes
    def count_nodes(self):
        if self.head is None:
            print("Number of nodes: 0")
            return

        count = 0
        temp = self.head

        while True:
            count += 1
            temp = temp.next

            if temp == self.head:
                break

        print("Number of nodes:", count)

    # 9(a). Display Head to Tail
    def display_head_to_tail(self):
        if self.head is None:
            print("Linked list is empty.")
            return

        temp = self.head

        print("Head to Tail:", end=" ")

        while True:
            print(temp.data, end=" -> ")
            temp = temp.next

            if temp == self.head:
                break

        print("(Head)")

    # 9(b). Display Tail to Head
    def display_tail_to_head(self):
```

```
        if self.head is None:
            print("Linked list is empty.")
            return

        # Store nodes in a list
        nodes = []
        temp = self.head

        while True:
            nodes.append(temp.data)
            temp = temp.next

            if temp == self.head:
                break

        print("Tail to Head:", end=" ")

        for i in range(len(nodes) - 1, -1, -1):
            print(nodes[i], end=" -> ")

        print("(Head)")


# Main Program

cll = CircularLinkedList()

while True:

    print("\n========== CIRCULAR LINKED LIST ==========")
    print("1. Create Linked List")
    print("2. Insert at Beginning")
    print("3. Insert at End")
    print("4. Insert at Index")
    print("5. Delete by Value")
    print("6. Delete First Node")
    print("7. Delete Last Node")
    print("8. Count Number of Nodes")
    print("9. Display")
    print("   a. Head to Tail")
    print("   b. Tail to Head")
    print("10. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        cll.create()

    elif choice == 2:
        cll.insert_beginning()
```

```
elif choice == 3:
    cll.insert_end()

elif choice == 4:
    cll.insert_at_index()

elif choice == 5:
    cll.delete_by_value()

elif choice == 6:
    cll.delete_first()

elif choice == 7:
    cll.delete_last()

elif choice == 8:
    cll.count_nodes()

elif choice == 9:
    print("\na. Head to Tail")
    print("b. Tail to Head")

    sub_choice = input("Enter your choice: ")

    if sub_choice == 'a':
        cll.display_head_to_tail()

    elif sub_choice == 'b':
        cll.display_tail_to_head()

    else:
        print("Invalid choice.")

elif choice == 10:
    print("Program exited.")
    break

else:
    print("Invalid choice. Please try again.")
```

OUTPUT:

```
==== RESTART: D:/CH ROHITH/DSA/WEEK 4 (Linked Lists)/Circular Linked List.py ===

========== CIRCULAR LINKED LIST ==========
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
   a. Head to Tail
   b. Tail to Head
10. Exit
Enter your choice: 2
Enter value: 31
Node inserted at beginning.

========== CIRCULAR LINKED LIST ==========
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
   a. Head to Tail
   b. Tail to Head
10. Exit
Enter your choice: 8
Number of nodes: 1
```

Explanation:

- Defines Node and CircularLinkedList classes where the last node points back to the head.
- Provides functions to create and insert nodes at the beginning, end and a specific index.
- Provides functions to delete nodes by value, from the beginning and from the end.
- Count and display functions traverse the list until they reach the head again.
- Additional functions display the head and tail and traverse the list from tail to head.
- A menu allows the user to perform different circular linked list operations.

Error Table:

S.No.	Errors Observed	Rectification
1)	Infinite loop while traversing the list	Stop traversal when the current node reaches the head
2)	Incorrect head after deletion	Update the head and last node link correctly
3)	Index out of range during insertion	Enter an index within the valid list range

WEEK-5

5A)

Aim:

Write a python program to implement a stack using an array, also implement the stack operations such as:
a)push
b)pop
c)peek also display the stack elements

PROGRAM:

```
stack = []
size = int(input("Enter stack size: "))

def push():
    if len(stack) >= size:
        print("Stack Overflow")
    else:
        element = int(input("Enter element: "))
        stack.append(element)
        print("Element pushed:", element)

def pop():
    if not stack:
        print("Stack Underflow")
    else:
        print("Popped element:", stack.pop())

def peek():
    if not stack:
        print("Stack is empty")
    else:
        print("Top element:", stack[-1])

def display():
    if not stack:
        print("Stack is empty")
    else:
        print("Stack elements:", stack[::-1])

while True:
    print("\n1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")
```

```
    choice = input("Enter your choice: ")

    if choice == "1":
        push()
    elif choice == "2":
        pop()
    elif choice == "3":
        peek()
    elif choice == "4":
        display()
    elif choice == "5":
        print("Program terminated")
        break
    else:
        print("Invalid choice")
```

OUTPUT:

```
========== RESTART: D:/CH ROHITH/DSA/WEEK 5()/STACK OPERATIONS(5A).py ==========
Enter stack size: 4

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 10
Element pushed: 10

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack elements: [10]
```

Explanation:

- Defines a StackEx class that creates a stack with a fixed size and initializes top as -1.
- The push function adds an element to the top and checks for stack overflow.
- The pop function removes the top element and checks for stack underflow.
- The peek function displays the current top element without removing it.
- The display function prints all stack elements from top to bottom.
- A menu allows the user to perform push, pop, peek, display and exit operations.

Error Table:

S.No.	Errors Observed	Rectification
1)	Stack Overflow occurs when adding elements	Increase the stack size or remove existing elements
2)	Stack Underflow occurs while popping	Check whether the stack contains elements before popping
3)	IndentationError in class or menu section	Maintain proper indentation throughout the program

5B)

Aim:

Write a python program to implement a stack using linked list ,also implement the stack operations such as:
a)push
b)pop
c)peek also display the stack elements

PROGRAM:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class Stack:
    def __init__(self):
        self.top = None

    def push(self, data):
        new_node = Node(data)
        new_node.next = self.top
        self.top = new_node
        print("Element pushed:", data)

    def pop(self):
        if self.top is None:
            print("Stack Underflow")
        else:
            print("Popped element:", self.top.data)
            self.top = self.top.next

    def peek(self):
        if self.top is None:
            print("Stack is empty")
```

```
        else:
            print("Top element:", self.top.data)

    def display(self):
        if self.top is None:
            print("Stack is empty")
        else:
            current = self.top
            print("Stack elements:")
            while current is not None:
                print(current.data)
                current = current.next

s = Stack()

while True:
    print("\n1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        element = int(input("Enter element: "))
        s.push(element)
    elif choice == "2":
        s.pop()
    elif choice == "3":
        s.peek()
    elif choice == "4":
        s.display()
    elif choice == "5":
        print("Program terminated")
        break
    else:
        print("Invalid choice")
```

OUTPUT:

```
Enter your choice:
======= RESTART: D:/CH ROHITH/DSA/WEEK 5()/Stack operations(using LL).py =======

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 45
Element pushed: 45

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack elements:
45
```

Explanation:

- Defines a Node class to store data and the link to the next node.
- The Stack class uses top to keep track of the top node.
- The push function creates a new node and adds it to the top of the stack.
- The pop function removes the top node and handles stack underflow.
- The peek and display functions show the top element and all stack elements.
- A menu allows the user to perform different stack operations.

Error Table:

S.No.	Errors Observed	Rectification
1)	Stack Underflow while popping	Check whether the stack is empty before popping
2)	AttributeError while accessing node data	Check that top points to a valid node
3)	IndentationError in class or menu section	Maintain proper indentation throughout the program

WEEK-6

6A)

Aim:

Write a python program to implement Queue and its operations such as enqueue, dequeue, peek, display with a menu-driven approach.Implement using array data structures.

PROGRAM:

```
queue = []
size = int(input("Enter queue size: "))

def enqueue():
    if len(queue) >= size:
        print("Queue Overflow")
    else:
        element = int(input("Enter element: "))
        queue.append(element)
        print("Element inserted:", element)

def dequeue():
    if len(queue) == 0:
        print("Queue Underflow")
    else:
        print("Deleted element:", queue.pop(0))

def peek():
    if len(queue) == 0:
        print("Queue is empty")
    else:
        print("Front element:", queue[0])

def display():
    if len(queue) == 0:
        print("Queue is empty")
    else:
        print("Queue elements:", queue)

while True:
    print("\n1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = input("Enter your choice: ")
```

```
    if choice == "1":
        enqueue()
    elif choice == "2":
        dequeue()
    elif choice == "3":
        peek()
    elif choice == "4":
        display()
    elif choice == "5":
        print("Program terminated")
        break
    else:
        print("Invalid choice")
```

OUTPUT:

```
=== RESTART: D:/CH ROHITH/DSA/WEEK 6(Queue)/Queue Operations(using arrays).py ==
Enter queue size: 4

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 45
Element inserted: 45

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Front element: 45
```

Explanation:

- Defines a Queue class using a fixed size list and initializes front and rear as -1.
- The enqueue function adds elements at the rear and checks for queue overflow.
- The dequeue function removes elements from the front and checks for queue underflow.
- The peek function displays the front element without removing it.
- The display function prints all elements from front to rear.
- A menu allows the user to perform enqueue, dequeue, peek, display and exit operations.

Error Table:

S.No.	Errors Observed	Rectification
1)	Queue Overflow when inserting an element	Increase the queue size or remove existing elements

2)	Queue Underflow while deleting an element	Check whether the queue contains elements before deletion
3)	Elements are not displayed correctly	Check the front and rear values before displaying

6B)

Aim:

Write a python program to implement Queue and its operations such as enqueue, dequeue, peek, display with a menu-driven approach.Implement using linked list data structures.

PROGRAM:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class Queue:
    def __init__(self):
        self.front = None
        self.rear = None

    def enqueue(self, data):
        new_node = Node(data)

        if self.rear is None:
            self.front = self.rear = new_node
        else:
            self.rear.next = new_node
            self.rear = new_node

        print("Element inserted:", data)

    def dequeue(self):
        if self.front is None:
            print("Queue Underflow")
        else:
            print("Deleted element:", self.front.data)
            self.front = self.front.next

            if self.front is None:
                self.rear = None

    def peek(self):
```

```
        if self.front is None:
            print("Queue is empty")
        else:
            print("Front element:", self.front.data)

    def display(self):
        if self.front is None:
            print("Queue is empty")
        else:
            current = self.front
            print("Queue elements:", end=" ")

            while current is not None:
                print(current.data, end=" ")
                current = current.next

            print()


q = Queue()

while True:
    print("\n1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        element = int(input("Enter element: "))
        q.enqueue(element)
    elif choice == "2":
        q.dequeue()
    elif choice == "3":
        q.peek()
    elif choice == "4":
        q.display()
    elif choice == "5":
        print("Program terminated")
        break
    else:
        print("Invalid choice")
```

OUTPUT:

```
Enter your choice:
===== RESTART: D:/CH ROHITH/DSA/WEEK 6(Queue)/Queue Operations(using LL).py ====

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 45
Element inserted: 45

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Queue elements: 45
```

Explanation:

- Defines a Node class to store the queue element and the link to the next node.
- QueueEx uses front and rear pointers to manage the queue.
- The enqueue function adds a new node at the rear of the queue.
- The dequeue function removes a node from the front and handles an empty queue.
- The peek and display functions show the front element and all queue elements.
- A menu allows the user to perform enqueue, dequeue, peek, display and exit operations.

Error Table:

S.No.	Errors Observed	Rectification
1)	Queue Underflow during dequeue	Check whether front is None before deleting
2)	Rear pointer becomes incorrect	Set rear to None when the last node is removed
3)	Elements are not displayed correctly	Check the next link while traversing the queue

6C)

Aim:

Write a python program to implement Circular Queue and its operations such as enqueue, dequeue, peek, display with a menu-driven approach.

PROGRAM:

```
queue = []
size = int(input("Enter queue size: "))
front = -1
rear = -1

def enqueue():
    global front, rear

    if (rear + 1) % size == front:
        print("Circular Queue Overflow")
        return

    element = int(input("Enter element: "))

    if front == -1:
        front = 0

    rear = (rear + 1) % size
    if len(queue) < size:
        queue.append(None)
    queue[rear] = element

    print("Element inserted:", element)

def dequeue():
    global front, rear

    if front == -1:
        print("Circular Queue Underflow")
        return

    print("Deleted element:", queue[front])

    if front == rear:
        front = -1
        rear = -1
    else:
        front = (front + 1) % size

def peek():
    if front == -1:
        print("Circular Queue is empty")
    else:
        print("Front element:", queue[front])

def display():
    if front == -1:
        print("Circular Queue is empty")
        return
```

```
    print("Circular Queue elements:", end=" ")
    i = front

    while True:
        print(queue[i], end=" ")
        if i == rear:
            break
        i = (i + 1) % size

    print()

while True:
    print("\n1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        enqueue()
    elif choice == "2":
        dequeue()
    elif choice == "3":
        peek()
    elif choice == "4":
        display()
    elif choice == "5":
        print("Program terminated")
        break
    else:
        print("Invalid choice")
```

OUTPUT:

```
===== RESTART: D:/CH ROHITH/DSA/WEEK 6(Queue)/Circular Queue Operations.py =====
Enter queue size: 5

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 24
Element inserted: 24

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Circular Queue elements: 24
```

Explanation:

- Defines a CircularQueue class using a fixed size list with front and rear pointers.
- The enqueue function inserts elements and uses modulo to move rear circularly.
- The dequeue function removes elements from the front and updates the front position.
- The queue checks for overflow when the next rear position reaches the front.
- The display function starts from front and moves circularly until it reaches rear.
- A menu allows the user to perform enqueue, dequeue, display and exit operations.

Error Table:

S.No.	Errors Observed	Rectification
1)	Queue Overflow occurs unexpectedly	Check the circular full condition using front and rear
2)	Queue Underflow while deleting	Check whether front is set to -1 before deletion
3)	Incorrect elements are displayed	Check the circular movement using modulo operation

WEEK – 7

7A)

Aim:

Write a Python Program to implement a binary tree using an array with all of its operations

PROGRAM:

```
tree = [None] * 100

def insert():
    x = int(input("Enter value: "))

    if tree[0] is None:
        tree[0] = x
        print("Node inserted")
        return

    p = int(input("Enter parent index: "))
    if p < 0 or p >= 100 or tree[p] is None:
        print("Invalid parent")
        return

    ch = input("Enter L for left or R for right: ").upper()

    if ch == 'L':
        i = 2 * p + 1
    elif ch == 'R':
        i = 2 * p + 2
    else:
        print("Invalid choice")
        return

    if i >= 100:
        print("Tree is full")
    elif tree[i] is not None:
        print("Position already occupied")
    else:
        tree[i] = x
        print("Node inserted")


def search():
    x = int(input("Enter value to search: "))
```

```
    for i in range(100):
        if tree[i] == x:
            print("Node found at index", i)
            return
    print("Node not found")


def delete():
    x = int(input("Enter value to delete: "))

    for i in range(100):
        if tree[i] == x:
            tree[i] = None
            print("Node deleted")
            return

    print("Node not found")


def display():
    print("Array representation:")
    print(tree)


def preorder(i):
    if i < 100 and tree[i] is not None:
        print(tree[i], end=" ")
        preorder(2 * i + 1)
        preorder(2 * i + 2)


def inorder(i):
    if i < 100 and tree[i] is not None:
        inorder(2 * i + 1)
        print(tree[i], end=" ")
        inorder(2 * i + 2)


def postorder(i):
    if i < 100 and tree[i] is not None:
        postorder(2 * i + 1)
        postorder(2 * i + 2)
        print(tree[i], end=" ")


while True:
    print("\n\n1.Insert")
    print("2.Delete")
```

```
print("3.Search")
print("4.Display")
print("5.Preorder")
print("6.Inorder")
print("7.Postorder")
print("8.Exit")

ch = int(input("Enter choice: "))

if ch == 1:
    insert()
elif ch == 2:
    delete()
elif ch == 3:
    search()
elif ch == 4:
    display()
elif ch == 5:
    preorder(0)
elif ch == 6:
    inorder(0)
elif ch == 7:
    postorder(0)
elif ch == 8:
    break
else:
    print("Invalid choice")
```

OUTPUT:

```
======= RESTART: D:/CH ROHITH/DSA/WEEK 7(Binary Tree)/Basic Operations.py ======


1.Insert
2.Delete
3.Search
4.Display
5.Preorder
6.Inorder
7.Postorder
8.Exit
Enter choice: 1
Enter value: 24
Node inserted


1.Insert
2.Delete
3.Search
4.Display
5.Preorder
6.Inorder
7.Postorder
8.Exit
Enter choice: 4
Array representation:
[24, None, None, None, None, None, None, None, None, None, None, None, None, Non
```

Explanation:

- Defines a Node class to store the queue element and the link to the next node.
- Array Representation: The binary tree is stored in an array, where the root node is placed at index 0. For a node at index i, its left child is at 2i + 1 and the right child is at 2i + 2.
- Insertion: The insert () function adds elements to the tree. In this program, smaller values are placed in the left subtree and larger values in the right subtree.
- Tree Traversals: The program supports order, preorder, post order, and level-order traversal to visit all nodes in different orders.
- Searching and Counting: The program can search for a particular value and calculate the total number of nodes and leaf nodes in the tree.
- Other Operations: The program calculates the height of the tree and displays the array of representation. A menu-driven approach allows the user to select different operations.

Error Table:

S.No.	Errors Observed	Rectification
1)	Using the wrong index for the right child.	Use 2 * index + 2 for the right child.
2)	Accessing an array index beyond MAX. Check if index >= MAX before accessing the array.	Accessing an array index beyond MAX. Check if index >= MAX before accessing the array.
3)	Forgetting to initialize the array.	Initialize it using tree = [None] * MAX.

7B)

Aim:

Write a Python Program to implement a binary tree using an linked list with all of its operations

PROGRAM:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None


root = None
```

```
def insert():
    global root
    x = int(input("Enter value: "))
    n = Node(x)

    if root is None:
        root = n
        print("Node inserted")
        return

    q = [root]

    while q:
        t = q.pop(0)

        if t.left is None:
            t.left = n
            print("Node inserted")
            return
        else:
            q.append(t.left)

        if t.right is None:
            t.right = n
            print("Node inserted")
            return
        else:
            q.append(t.right)


def search(t, x):
    if t is None:
        return None
    if t.data == x:
        return t

    a = search(t.left, x)
    if a:
        return a

    return search(t.right, x)


def delete():
    global root
    x = int(input("Enter value to delete: "))
```

```
    if root is None:
        print("Tree is empty")
        return

    q = [(root, None, None)]
    target = None
    last = None

    while q:
        t, p, side = q.pop(0)

        if t.data == x and target is None:
            target = (t, p, side)

        last = (t, p, side)

        if t.left:
            q.append((t.left, t, 'L'))
        if t.right:
            q.append((t.right, t, 'R'))

    if target is None:
        print("Node not found")
        return

    t, p, side = target
    last_node, last_parent, last_side = last

    t.data = last_node.data

    if last_parent is None:
        root = None
    elif last_side == 'L':
        last_parent.left = None
    else:
        last_parent.right = None

    print("Node deleted")


def display(t):
    if t:
        print(t.data, end=" ")
        display(t.left)
        display(t.right)


def preorder(t):
```

```
    if t:
        print(t.data, end=" ")
        preorder(t.left)
        preorder(t.right)


def inorder(t):
    if t:
        inorder(t.left)
        print(t.data, end=" ")
        inorder(t.right)


def postorder(t):
    if t:
        postorder(t.left)
        postorder(t.right)
        print(t.data, end=" ")


while True:
    print("\n\n1.Insert")
    print("2.Delete")
    print("3.Search")
    print("4.Display")
    print("5.Preorder")
    print("6.Inorder")
    print("7.Postorder")
    print("8.Exit")

    ch = int(input("Enter choice: "))

    if ch == 1:
        insert()
    elif ch == 2:
        delete()
    elif ch == 3:
        x = int(input("Enter value to search: "))
        if search(root, x):
            print("Node found")
        else:
            print("Node not found")
    elif ch == 4:
        print("Tree elements:")
        display(root)
    elif ch == 5:
        preorder(root)
    elif ch == 6:
```

```
        inorder(root)
    elif ch == 7:
        postorder(root)
    elif ch == 8:
        break
    else:
        print("Invalid choice")
```

OUTPUT:

```
== RESTART: D:/CH ROHITH/DSA/WEEK 7(Binary Tree)/Basic Operations(Using LL).py =


1.Insert
2.Delete
3.Search
4.Display
5.Preorder
6.Inorder
7.Postorder
8.Exit
Enter choice: 1
Enter value: 24
Node inserted


1.Insert
2.Delete
3.Search
4.Display
5.Preorder
6.Inorder
7.Postorder
8.Exit
Enter choice: 6
24
```

Explanation:

- Defines a Node class to store the queue element and the link to the next node.
- Node Structure: Each tree node is created using a Node class containing three parts: data, left pointer, and right pointer.
- Insertion: The insert() function adds nodes using level-order insertion, filling the left and right positions of each level before moving to the next level.
- Tree Traversals: The program provides inorder, preorder, postorder, and level-order traversal to display the elements of the binary tree.
- Searching and Counting: The program can search for a value and calculate the total number of nodes and leaf nodes using recursive functions.
- Additional Operations: The program calculates the height of the tree and finds the minimum and maximum values. A menu-driven system allows the user to perform each operation easily.

Error Table:

S.No.	Errors Observed	Rectification
1)	Creating a node without initializing its child pointers.	Set self.left = None and self.right = None in the constructor.
2)	Forgetting to set the root when the tree is empty.	Use if self.root is None: self.root = new_node.
3)	Calling traversal without passing the root node.	Call it as tree.inorder(tree.root), tree.preorder(tree.root), etc.

WEEK – 8

8A)

Aim:

Write a Python program to implement a Binary Search Tree (BST)

PROGRAM:

```
tree = []
def insert(value, i=0):
    global tree

    if i >= len(tree):
        tree.extend([None] * (i - len(tree) + 1))

    if tree[i] is None:
        tree[i] = value
    elif value < tree[i]:
        insert(value, 2 * i + 1)
    elif value > tree[i]:
        insert(value, 2 * i + 2)
    else:
        print("Duplicate values are not allowed.")


def search(value, i=0):
    if i >= len(tree) or tree[i] is None:
        return False

    if tree[i] == value:
        return True
    elif value < tree[i]:
        return search(value, 2 * i + 1)
    else:
        return search(value, 2 * i + 2)


def minimum(i):
    while 2 * i + 1 < len(tree) and tree[2 * i + 1] is not None:
        i = 2 * i + 1
    return i


def delete(value, i=0):
    if i >= len(tree) or tree[i] is None:
        return
```

```
    if value < tree[i]:
        delete(value, 2 * i + 1)
    elif value > tree[i]:
        delete(value, 2 * i + 2)
    else:
        left = 2 * i + 1
        right = 2 * i + 2

        has_left = left < len(tree) and tree[left] is not None
        has_right = right < len(tree) and tree[right] is not None

        if not has_left and not has_right:
            tree[i] = None

        elif not has_left:
            tree[i] = tree[right]
            delete_subtree(right, i)

        elif not has_right:
            tree[i] = tree[left]
            delete_subtree(left, i)

        else:
            j = minimum(right)
            tree[i] = tree[j]
            delete(tree[j], right)


def delete_subtree(child, parent):
    if child >= len(tree) or tree[child] is None:
        return

    value = tree[child]
    delete(value, child)

    remaining = collect_subtree(child)
    for v in remaining:
        insert(v)

def collect_subtree(i):
    if i >= len(tree) or tree[i] is None:
        return []
    value = tree[i]
    left = collect_subtree(2 * i + 1)
    right = collect_subtree(2 * i + 2)
    tree[i] = None
    return left + [value] + right
```

```
def inorder(i=0):
    if i < len(tree) and tree[i] is not None:
        inorder(2 * i + 1)
        print(tree[i], end=" ")
        inorder(2 * i + 2)


def preorder(i=0):
    if i < len(tree) and tree[i] is not None:
        print(tree[i], end=" ")
        preorder(2 * i + 1)
        preorder(2 * i + 2)


def postorder(i=0):
    if i < len(tree) and tree[i] is not None:
        postorder(2 * i + 1)
        postorder(2 * i + 2)
        print(tree[i], end=" ")


while True:
    print("\n\n1. Insert")
    print("2. Delete")
    print("3. Search")
    print("4. Inorder Traversal")
    print("5. Preorder Traversal")
    print("6. Postorder Traversal")
    print("7. Display Array")
    print("8. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        value = int(input("Enter value: "))
        insert(value)

    elif choice == 2:
        value = int(input("Enter value to delete: "))
        if search(value):
            delete(value)
            print("Value deleted.")
        else:
            print("Value not found.")

    elif choice == 3:
        value = int(input("Enter value to search: "))
        print("Found" if search(value) else "Not found")

    elif choice == 4:
```

```
        inorder()
        print()

    elif choice == 5:
        preorder()
        print()

    elif choice == 6:
        postorder()
        print()

    elif choice == 7:
        print(tree)

    elif choice == 8:
        break

    else:
        print("Invalid choice.")
```

OUTPUT:

```
====================== RESTART: D:/CH ROHITH/DSA/bst.py ======================


1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Array
8. Exit
Enter your choice: 1
Enter value: 24


1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Array
8. Exit
Enter your choice: 7
[24]
```

Explanation:

Defines a Binary Search Tree (BST) using an array to store the elements.
Inserts elements according to the BST property: smaller values go to the left, and larger values go to the right.
Uses index 0 to store the root node.

For a node at index i, the left child is at index 2*i + 1 and the right child is at index 2*i + 2.
Performs insertion, deletion, searching, and traversal using a menu-driven program.
Displays the elements using inorder, preorder, and postorder traversal.

Error Table:

S.No.	Errors Observed	Rectification
1)	Array index out of bounds	Extend the array when a required index exceeds its size.
2)	Incorrect insertion position	Compare values and use the correct left and right child indices.
3)	Element not found during deletion	Search for the element before deleting it.

8B)

AIM:

Write a Python program to implement a Binary Search Tree (BST) use Ll

PROGRAM:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

def insert(root, value):
    if root is None:
        return Node(value)

    if value < root.data:
        root.left = insert(root.left, value)
    elif value > root.data:
        root.right = insert(root.right, value)
    else:print("Duplicate values are not allowed.")

    return root

def search(root, value):
    if root is None:
        return False
```

```
    if root.data == value:
        return True
    elif value < root.data:
        return search(root.left, value)
    else:
        return search(root.right, value)

def find_min(root):
    while root.left is not None:
        root = root.left
    return root

def delete(root, value):
    if root is None:
        return None

    if value < root.data:
        root.left = delete(root.left, value)

    elif value > root.data:
        root.right = delete(root.right, value)

    else:
        # No left child
        if root.left is None:
            return root.right

        # No right child
        if root.right is None:
            return root.left

        # Two children
        temp = find_min(root.right)
        root.data = temp.data
        root.right = delete(root.right, temp.data)

    return root

def inorder(root):
    if root:
        inorder(root.left)
        print(root.data, end=" ")
        inorder(root.right)

def preorder(root):
    if root:
        print(root.data, end=" ")
        preorder(root.left)
        preorder(root.right)
```

```
def postorder(root):
    if root:
        postorder(root.left)
        postorder(root.right)
        print(root.data, end=" ")

root = None

while True:
    print("\n\n1. Insert")
    print("2. Delete")
    print("3. Search")
    print("4. Inorder Traversal")
    print("5. Preorder Traversal")
    print("6. Postorder Traversal")
    print("7. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        value = int(input("Enter value: "))
        root = insert(root, value)

    elif choice == 2:
        value = int(input("Enter value to delete: "))
        if search(root, value):
            root = delete(root, value)
            print("Value deleted.")
        else:
            print("Value not found.")

    elif choice == 3:
        value = int(input("Enter value to search: "))
        if search(root, value):
            print("Value found.")
        else:
            print("Value not found.")

    elif choice == 4:
        inorder(root)
        print()

    elif choice == 5:
        preorder(root)
        print()

    elif choice == 6:
        postorder(root)
        print()
```

```
    elif choice == 7:
        break

    else:
        print("Invalid choice.")
```

OUTPUT:

```
============ RESTART: D:/CH ROHITH/DSA/WEEK 8(BST)/bst(Using LL).py ============


1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter your choice: 1
Enter value: 24


1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter your choice: 4
24
```

EXPLANATION:

Defines a Binary Search Tree (BST) using a linked list structure.
Each node contains three fields: data, left, and right.
Inserts smaller values into the left subtree and larger values into the right subtree.
Uses a root pointer to represent the starting node of the tree.
Performs insertion, deletion, searching, and traversal using a menu-driven program.
Displays the elements using inorder, preorder, and postorder traversal.

ERROR TABLE:

S.No	Error Obtained	Rectification
1)	Null pointer error	Check whether the node is None before accessing its data.
2)	Incorrect insertion position	Compare the value with the current node and follow the correct pointer.
3	Incorrect deletion of a node	Handle leaf nodes, nodes with one child, and nodes with two children separately.

WEEK 9

9A)

AIM:

Write a Python program to implement a AVL tree using arrasy

PROGRAM:

```
keys = []
left = []
right = []
heights = []
root = -1

def new_node(key):
    i = len(keys)
    keys.append(key)
    left.append(-1)
    right.append(-1)
    heights.append(1)
    return i

def height(i):
    return 0 if i == -1 else heights[i]

def update(i):
    heights[i] = 1 + max(height(left[i]), height(right[i]))

def balance(i):
    return 0 if i == -1 else height(left[i]) - height(right[i])

def rotate_right(y):
    x = left[y]
    t = right[x]

    right[x] = y
    left[y] = t

    update(y)
    update(x)
    return x

def rotate_left(x):
    y = right[x]
    t = left[y]
```

```
    left[y] = x
    right[x] = t

    update(x)
    update(y)
    return y

def insert(node, key):
    if node == -1:
        return new_node(key)

    if key < keys[node]:
        left[node] = insert(left[node], key)
    elif key > keys[node]:
        right[node] = insert(right[node], key)
    else:
        return node

    update(node)
    bf = balance(node)

    if bf > 1 and key < keys[left[node]]:
        return rotate_right(node)

    if bf < -1 and key > keys[right[node]]:
        return rotate_left(node)

    if bf > 1 and key > keys[left[node]]:
        left[node] = rotate_left(left[node])
        return rotate_right(node)

    if bf < -1 and key < keys[right[node]]:
        right[node] = rotate_right(right[node])
        return rotate_left(node)

    return node

def min_node(node):
    while left[node] != -1:
        node = left[node]
    return node

def delete(node, key):
    if node == -1:
        return -1

    if key < keys[node]:
        left[node] = delete(left[node], key)
    elif key > keys[node]:
```

```
            right[node] = delete(right[node], key)
        else:
            if left[node] == -1:
                return right[node]
            if right[node] == -1:
                return left[node]

            temp = min_node(right[node])
            keys[node] = keys[temp]
            right[node] = delete(right[node], keys[temp])

        update(node)
        bf = balance(node)

        if bf > 1 and balance(left[node]) >= 0:
            return rotate_right(node)

        if bf > 1 and balance(left[node]) < 0:
            left[node] = rotate_left(left[node])
            return rotate_right(node)

        if bf < -1 and balance(right[node]) <= 0:
            return rotate_left(node)

        if bf < -1 and balance(right[node]) > 0:
            right[node] = rotate_right(right[node])
            return rotate_left(node)

        return node

    def search(node, key):
        while node != -1:
            if keys[node] == key:
                return True
            if key < keys[node]:
                node = left[node]
            else:
                node = right[node]
        return False

    def inorder(node):
        if node != -1:
            inorder(left[node])
            print(keys[node], end=" ")
            inorder(right[node])

    def preorder(node):
        if node != -1:
            print(keys[node], end=" ")
            preorder(left[node])
```

```
        preorder(right[node])

def postorder(node):
    if node != -1:
        postorder(left[node])
        postorder(right[node])
        print(keys[node], end=" ")

while True:
    print("\n\n--- AVL TREE USING ARRAYS ---")
    print("1. Insert")
    print("2. Delete")
    print("3. Search")
    print("4. Inorder")
    print("5. Preorder")
    print("6. Postorder")
    print("7. Exit")

    choice = input("Enter choice: ")

    if choice == '1':
        key = int(input("Enter value: "))
        root = insert(root, key)
        print("Node inserted.")

    elif choice == '2':
        key = int(input("Enter value: "))
        if search(root, key):
            root = delete(root, key)
            print("Node deleted.")
        else:
            print("Value not found.")

    elif choice == '3':
        key = int(input("Enter value: "))
        print("Value found." if search(root, key)
              else "Value not found.")

    elif choice == '4':
        inorder(root)
        print()

    elif choice == '5':
        preorder(root)
        print()

    elif choice == '6':
        postorder(root)
        print()
```

```
    elif choice == '7':
        break

    else:
        print("Invalid choice.")
```

OUTPUT:

```
========== RESTART: D:/CH ROHITH/DSA/WEEK 9/AVL.py ==========


--- AVL TREE USING ARRAYS ---
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Exit
Enter choice: 1
Enter value: 24
Node inserted.


--- AVL TREE USING ARRAYS ---
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Exit
Enter choice: 4
24
```

EXPLANATION:

Creates arrays to store node values, left-child indices, right-child indices, and node heights.
Inserts elements according to Binary Search Tree (BST) properties.
Calculates the height and balance factor of each node.
Performs LL, RR, LR, and RL rotations to maintain AVL tree balance.
Deletes nodes and rebalances the tree whenever necessary.
Searches for an element and displays whether it is found or not.
Performs inorder, preorder, and postorder traversals.

ERROR TABLE:

S.No	Error Obtained	Rectification
1	Incorrect child index	Verify left and right child indices are updated correctly during rotations.
2	Tree becomes unbalanced	Calculate the balance factor and apply the appropriate rotation.
3	Incorrect height calculation	Update node heights after insertion, deletion, and rotations.

9B)

AIM:

Write a Python program to implement a AVL tree using linked list

PROGRAM:

```
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
        self.height = 1

def height(root):
    return 0 if root is None else root.height

def balance(root):
    if root is None:
        return 0
    return height(root.left) - height(root.right)

def update(root):
    root.height = 1 + max(height(root.left), height(root.right))

def rotate_right(y):
    x = y.left
    t = x.right

    x.right = y
    y.left = t

    update(y)
    update(x)
```

```
    return x

def rotate_left(x):
    y = x.right
    t = y.left

    y.left = x
    x.right = t

    update(x)
    update(y)
    return y

def insert(root, key):
    if root is None:
        return Node(key)

    if key < root.key:
        root.left = insert(root.left, key)
    elif key > root.key:
        root.right = insert(root.right, key)
    else:
        return root

    update(root)
    bf = balance(root)

    if bf > 1 and key < root.left.key:
        return rotate_right(root)

    if bf < -1 and key > root.right.key:
        return rotate_left(root)

    if bf > 1 and key > root.left.key:
        root.left = rotate_left(root.left)
        return rotate_right(root)

    if bf < -1 and key < root.right.key:
        root.right = rotate_right(root.right)
        return rotate_left(root)

    return root

def min_node(root):
    while root.left is not None:
        root = root.left
    return root

def delete(root, key):
    if root is None:
```

```
            return None

        if key < root.key:
            root.left = delete(root.left, key)
        elif key > root.key:
            root.right = delete(root.right, key)
        else:
            if root.left is None:
                return root.right
            elif root.right is None:
                return root.left

            temp = min_node(root.right)
            root.key = temp.key
            root.right = delete(root.right, temp.key)

        update(root)
        bf = balance(root)

        if bf > 1 and balance(root.left) >= 0:
            return rotate_right(root)

        if bf > 1 and balance(root.left) < 0:
            root.left = rotate_left(root.left)
            return rotate_right(root)

        if bf < -1 and balance(root.right) <= 0:
            return rotate_left(root)

        if bf < -1 and balance(root.right) > 0:
            root.right = rotate_right(root.right)
            return rotate_left(root)

        return root

    def search(root, key):
        if root is None or root.key == key:
            return root

        if key < root.key:
            return search(root.left, key)

        return search(root.right, key)

    def inorder(root):
        if root:
            inorder(root.left)
            print(root.key, end=" ")
            inorder(root.right)
```

```
def preorder(root):
    if root:
        print(root.key, end=" ")
        preorder(root.left)
        preorder(root.right)

def postorder(root):
    if root:
        postorder(root.left)
        postorder(root.right)
        print(root.key, end=" ")

root = None

while True:
    print("\n\n--- AVL TREE USING LINKED LIST ---")
    print("1. Insert")
    print("2. Delete")
    print("3. Search")
    print("4. Inorder")
    print("5. Preorder")
    print("6. Postorder")
    print("7. Exit")

    choice = input("Enter choice: ")

    if choice == '1':
        key = int(input("Enter value: "))
        root = insert(root, key)
        print("Node inserted.")

    elif choice == '2':
        key = int(input("Enter value: "))
        if search(root, key):
            root = delete(root, key)
            print("Node deleted.")
        else:
            print("Value not found.")

    elif choice == '3':
        key = int(input("Enter value: "))
        if search(root, key):
            print("Value found.")
        else:
            print("Value not found.")

    elif choice == '4':
        inorder(root)
        print()
```

```
    elif choice == '5':
        preorder(root)
        print()

    elif choice == '6':
        postorder(root)
        print()

    elif choice == '7':
        print("Exiting program.")
        break

    else:
        print("Invalid choice.")
```

OUTPUT:

```
=============== RESTART: D:/CH ROHITH/DSA/WEEK 9/AVL(Using LL).py ==============


--- AVL TREE USING LINKED LIST ---
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Exit
Enter choice: 1
Enter value: 24
Node inserted.


--- AVL TREE USING LINKED LIST ---
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Exit
Enter choice: 4
24
```

EXPLANATION:

Creates nodes containing a key, left-child reference, right-child reference, and height.
Inserts elements according to Binary Search Tree (BST) properties.
Calculates the height and balance factor of each node.

Performs LL, RR, LR, and RL rotations to maintain AVL tree balance.
Deletes nodes and restores balance using rotations.
Searches for an element by comparing the target with each node's key.
Performs inorder, preorder, and postorder traversals.
Displays whether the searched element is found or not.

ERROR TABLE:

S.No	Error Obtained	Rectification
1	AttributeError while accessing a node	Check whether the node is None before accessing its attributes.
2	Tree becomes unbalanced	Calculate the balance factor and apply the correct rotation.
3	Incorrect height calculation	Update the heights of affected nodes after every rotation.

WEEK 10

10A)

AIM:

Python programs for hashing using separate chaining, linear probing, and quadratic probing.

PROGRAM:

```
size = int(input("Enter table size: "))
table = [[] for _ in range(size)]

def insert(key):
    table[key % size].append(key)

def search(key):
    return key in table[key % size]

def display():
    for index, bucket in enumerate(table):
        print(index, bucket)

for key in map(int, input("Enter keys: ").split()):
    insert(key)

display()
target = int(input("Enter key to search: "))
print("Found" if search(target) else "Not found")
```

OUTPUT:

```
======= RESTART: G:\CH ROHITH\WEEK 10\10(1)_hashing_separate_chaining.py =======
Enter table size: 10
Enter keys: 1 3 2 5 6
0 []
1 [1]
2 [2]
3 [3]
4 []
5 [5]
6 [6]
7 []
8 []
9 []
Enter key to search: 2
Found
```

EXPLANATION:

•The size = 10 statement creates a hash table with 10 positions.
•The table = [[] for i in range(size)] statement creates an empty list at every position.
•The insert() function calculates the index using key % size.
•If two keys have the same index, both keys are stored in the same list.
•The display() function prints each index and its stored elements.
•The for loop accepts keys from the user and inserts them into the hash table.

ERROR TABLE:

S.no	Errors	Rectified
1.	Adding incompatible data types.	Rectified
2.	Entering a non-integer key.	Rectified.

10B)

AIM:

Hashing using linear probing.

Program:

```
 size = int(input("Enter table size: "))
table = [None] * size

def insert(key):
    index = key % size
    for _ in range(size):
        if table[index] is None:
            table[index] = key
            return True
        index = (index + 1) % size
    return False

def search(key):
    index = key % size
    for _ in range(size):
        if table[index] is None:
```

```
            return False
        if table[index] == key:
            return True
        index = (index + 1) % size
    return False

for key in map(int, input("Enter keys: ").split()):
    if not insert(key):
        print("Hash table is full")

print(table)
target = int(input("Enter key to search: "))
print("Found" if search(target) else "Not found")
```

Output:

```
========= RESTART: G:\CH ROHITH\WEEK 10\10(2)_hashing_linear_probing.py ========
Enter table size: 10
Enter keys: 1 3 5 2 4 7
[None, 1, 2, 3, 4, 5, None, 7, None, None]
Enter key to search: 3
Found
```

Explanation:

•The size = 10 statement defines the size of the hash table.
•The table = [-1] * size statement creates an empty table, where -1 represents an empty position.
•The insert() function calculates the index using key % size.
•If the position is occupied, the while loop checks the next position using (index + 1) % size.
•The display() function prints all indices and their values.
•The for loop accepts keys from the user and inserts them into the hash table using linear probing.

Error table:

S.no	Errors	Rectified
1.	Using an undefined variable or function.	Rectified
2.	Entering invalid input.	Rectified.

10C)

AIM:

Hashing using quadratic probing.

Program:

```
 size = int(input("Enter table size: "))
table = [None] * size

def insert(key):
    base = key % size
    for i in range(size):
        index = (base + i * i) % size
        if table[index] is None:
            table[index] = key
            return True
    return False

def search(key):
    base = key % size
    for i in range(size):
        index = (base + i * i) % size
        if table[index] is None:
            return False
        if table[index] == key:
            return True
    return False

for key in map(int, input("Enter keys: ").split()):
    if not insert(key):
        print("Could not insert key")

print(table)
target = int(input("Enter key to search: "))
print("Found" if search(target) else "Not found")
```

Output:

```
======= RESTART: G:\CH ROHITH\WEEK 10\10(3)_hashing_quadratic_probing.py =======
Enter table size: 10
Enter keys: 1 3 6 2 5
[None, 1, 2, 3, None, 5, 6, None, None, None]
Enter key to search: 4
Not found
```

Explanation:

•The size = 10 statement defines the size of the hash table.
•The table = [-1] * size statement creates an empty table, where -1 indicates an empty position.
•The insert() function calculates the initial index using key % size.
•The while loop checks positions using (index + i*i) % size when a collision occurs.
•If an empty position is found, the key is inserted there; otherwise, the loop continues checking positions up to the table size.
•The display() function prints the hash table, while the for loop accepts keys from the user and inserts them.

Error table:

S.no	Errors	Rectified
1.	Using an undefined variable or function.	Rectified
2.	Keys are inserted successfully.	Rectified.